P01-03 · T4–T6 aura-catalog — Schema v1, Migrations, Single Writer & Locks

The catalog is the only place AURA stores truth. Everything else — previews, embeddings, renders — is a cache that can be deleted and rebuilt. That asymmetry drives every decision on this page: one writer, WAL, verified backups before migrations, and a refusal to open a catalog we do not fully understand.

1. `migrations/0001_init.sql` — FROZEN schema v1

```
-- AURA catalog schema v1. FROZEN. Additive migrations only;
any change to an
-- existing column requires an ADR and a new numbered migrati
on file.
--
-- Conventions used throughout:
--   * IDs are TEXT with a type prefix, e.g. 'pht_018f...'. T
yped in Rust.
--   * Timestamps are TEXT in RFC-3339 UTC with a Z suffix, s
ortable as text.
--   * Booleans are INTEGER 0/1 with a CHECK constraint.
--   * Enums are TEXT with a CHECK constraint, never integer
s, so a hex dump of
--     the catalog is readable during a support call.

PRAGMA foreign_keys = ON;
```

```
-------------------------------------------------------------
---------
-- Schema versioning and provenance
-------------------------------------------------------------
---------
CREATE TABLE schema_version (
  version        INTEGER NOT NULL PRIMARY KEY,
  applied_at     TEXT    NOT NULL,
  app_version    TEXT    NOT NULL,
  migration_hash TEXT    NOT NULL           -- blake3 of the
migration file
) STRICT;

CREATE TABLE catalog_meta (
  key   TEXT NOT NULL PRIMARY KEY,
  value TEXT NOT NULL
) STRICT;
-- Seeded rows: catalog_uuid, created_at, created_by_app_vers
ion, machine_hash.

-------------------------------------------------------------
---------
-- Projects: one wedding, one project
-------------------------------------------------------------
---------
CREATE TABLE project (
  project_id     TEXT NOT NULL PRIMARY KEY,
  name           TEXT NOT NULL,
  couple_label   TEXT,                       -- C2. Optiona
l, photographer's words
  event_date     TEXT,                       -- date only, n
o time
  timezone       TEXT NOT NULL DEFAULT 'UTC', -- IANA name, f
or timeline ordering
  status         TEXT NOT NULL DEFAULT 'active'
```

```
                CHECK (status IN ('active','archived','deliv
ered')),
  created_at     TEXT NOT NULL,
  updated_at     TEXT NOT NULL,
  deleted_at     TEXT                          -- soft delet
e, purge is explicit
) STRICT;

CREATE INDEX idx_project_status_date ON project(status, event
_date DESC);

-- Consent lives in its own table so it can be revoked and au
dited without
-- touching the project row, and so a JOIN is required to dis
cover any
-- permission. There is exactly one row per project, created
with the project.
CREATE TABLE project_consent (
  project_id            TEXT NOT NULL PRIMARY KEY REFERENCES
project(project_id) ON DELETE CASCADE,
  cloud_metadata        INTEGER NOT NULL DEFAULT 0 CHECK (clo
ud_metadata IN (0,1)),
  cloud_derived_imagery INTEGER NOT NULL DEFAULT 0 CHECK (clo
ud_derived_imagery IN (0,1)),
  cloud_full_imagery    INTEGER NOT NULL DEFAULT 0 CHECK (clo
ud_full_imagery IN (0,1)),
  telemetry             INTEGER NOT NULL DEFAULT 0 CHECK (tel
emetry IN (0,1)),
  improve_models        INTEGER NOT NULL DEFAULT 0 CHECK (imp
rove_models IN (0,1)),
  granted_at            TEXT,
  granted_by            TEXT,
  expires_at            TEXT,
  revoked_at            TEXT
) STRICT;
```

```
-- Append-only audit of every consent change. Never updated,
never deleted.
CREATE TABLE consent_ledger (
  entry_id   INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
  project_id TEXT    NOT NULL REFERENCES project(project_id)
ON DELETE CASCADE,
  changed_at TEXT    NOT NULL,
  field      TEXT    NOT NULL,
  old_value  INTEGER NOT NULL,
  new_value  INTEGER NOT NULL,
  actor      TEXT    NOT NULL DEFAULT 'user'
) STRICT;

-----------------------------------------------------------------
---------
-- Where the files came from
-----------------------------------------------------------------
---------
CREATE TABLE source_root (
  root_id       TEXT NOT NULL PRIMARY KEY,
  project_id    TEXT NOT NULL REFERENCES project(project_id)
ON DELETE CASCADE,
  abs_path      TEXT NOT NULL,              -- C2, local onl
y, never egressed
  volume_label  TEXT,
  volume_serial TEXT,                       -- lets us recog
nise a reconnected drive
  is_removable  INTEGER NOT NULL DEFAULT 0 CHECK (is_removabl
e IN (0,1)),
  added_at      TEXT NOT NULL,
  last_seen_at  TEXT,
  UNIQUE (project_id, abs_path)
) STRICT;

-----------------------------------------------------------------
---------
```

```
-- Photos: the logical unit. One photo may have several files
-- (RAW + camera JPEG + XMP sidecar).
-------------------------------------------------------------
---------
CREATE TABLE photo (
  photo_id      TEXT NOT NULL PRIMARY KEY,
  project_id    TEXT NOT NULL REFERENCES project(project_id)
ON DELETE CASCADE,
  capture_time  TEXT,                       -- from EXIF, UT
C, may be NULL
  capture_time_source TEXT NOT NULL DEFAULT 'unknown'
                CHECK (capture_time_source IN ('exif_origina
l','exif_digitized','file_mtime','unknown')),
  camera_make   TEXT,
  camera_model  TEXT,
  lens_model    TEXT,
  iso           INTEGER,
  aperture      REAL,
  shutter_us    INTEGER,                     -- microsecond
s, integer for exactness
  focal_len_mm  REAL,
  orientation   INTEGER NOT NULL DEFAULT 1 CHECK (orientation
BETWEEN 1 AND 8),
  width_px      INTEGER,
  height_px     INTEGER,
  -- Populated in PHASE-02 onward. Present now so no migratio
n is needed later.
  primary_file_id TEXT,
  created_at    TEXT NOT NULL,
  updated_at    TEXT NOT NULL
) STRICT;

CREATE INDEX idx_photo_project_time  ON photo(project_id, cap
ture_time);
CREATE INDEX idx_photo_project_id_pk ON photo(project_id, pho
to_id);
```

```
CREATE INDEX idx_photo_camera        ON photo(project_id, cam
era_model);

-----------------------------------------------------------------
---------
-- Files: content-addressed. This table is the dedupe mechani
sm.
-----------------------------------------------------------------
---------
CREATE TABLE photo_file (
  file_id       TEXT NOT NULL PRIMARY KEY,
  photo_id      TEXT NOT NULL REFERENCES photo(photo_id) ON D
ELETE CASCADE,
  project_id    TEXT NOT NULL REFERENCES project(project_id)
ON DELETE CASCADE,
  root_id       TEXT NOT NULL REFERENCES source_root(root_id)
ON DELETE RESTRICT,
  rel_path      TEXT NOT NULL,              -- relative to r
oot, forward slashes
  file_name     TEXT NOT NULL,
  ext           TEXT NOT NULL,              -- lower case, n
o dot
  kind          TEXT NOT NULL
                 CHECK (kind IN ('raw','jpeg','heif','tif
f','png','sidecar_xmp','video','other')),
  role          TEXT NOT NULL
                 CHECK (role IN ('primary','companion_jpe
g','sidecar','derivative')),
  size_bytes    INTEGER NOT NULL CHECK (size_bytes >= 0),
  content_hash  TEXT NOT NULL,              -- blake3 hex, 6
4 chars
  mtime         TEXT NOT NULL,
  -- Cheap change detector: if size and mtime match, skip reh
ashing on rescan.
  fast_key      TEXT NOT NULL,              -- '<size>:<mtim
e_ms>'
```

```
  first_seen_at TEXT NOT NULL,
  last_seen_at  TEXT NOT NULL,
  is_present    INTEGER NOT NULL DEFAULT 1 CHECK (is_present
IN (0,1)),
  UNIQUE (project_id, content_hash, rel_path),
  UNIQUE (root_id, rel_path)
) STRICT;

CREATE INDEX idx_file_hash        ON photo_file(project_id, c
ontent_hash);
CREATE INDEX idx_file_photo       ON photo_file(photo_id, rol
e);
CREATE INDEX idx_file_fast_key    ON photo_file(root_id, fast
_key);
CREATE INDEX idx_file_missing     ON photo_file(project_id, i
s_present) WHERE is_present = 0;

-----------------------------------------------------------------
---------
-- EXIF: the raw key/value truth, kept separately from the pr
omoted columns
-- on photo so that nothing is silently lost by our own parsi
ng choices.
-----------------------------------------------------------------
---------
CREATE TABLE exif (
  file_id  TEXT NOT NULL REFERENCES photo_file(file_id) ON DE
LETE CASCADE,
  tag      TEXT NOT NULL,
  value    TEXT NOT NULL,
  PRIMARY KEY (file_id, tag)
) STRICT, WITHOUT ROWID;

-- GPS is stored separately and is class C2. A wedding venue
location plus a
-- date identifies a real event, so it is opt-in per project
```

```
to use it at all.
CREATE TABLE photo_gps (
  photo_id   TEXT NOT NULL PRIMARY KEY REFERENCES photo(photo
_id) ON DELETE CASCADE,
  lat        REAL NOT NULL,
  lon        REAL NOT NULL,
  altitude_m REAL,
  source     TEXT NOT NULL DEFAULT 'exif'
) STRICT;

-----------------------------------------------------------------
---------
-- Preview cache index. Bytes live on disk under cache_dir; t
his table only
-- records what exists so a deleted cache is self-healing.
-----------------------------------------------------------------
---------
CREATE TABLE preview (
  photo_id     TEXT NOT NULL REFERENCES photo(photo_id) ON DE
LETE CASCADE,
  tier         INTEGER NOT NULL CHECK (tier IN (1,2,3)),
  rel_cache_path TEXT NOT NULL,
  width_px     INTEGER NOT NULL,
  height_px    INTEGER NOT NULL,
  source       TEXT NOT NULL CHECK (source IN ('embedded','de
coded')),
  bytes        INTEGER NOT NULL,
  built_at     TEXT NOT NULL,
  stage_version INTEGER NOT NULL DEFAULT 1,
  PRIMARY KEY (photo_id, tier)
) STRICT, WITHOUT ROWID;

-----------------------------------------------------------------
---------
-- Import runs and quarantine
-----------------------------------------------------------------
```

```
---------
CREATE TABLE import_run (
  import_id       TEXT NOT NULL PRIMARY KEY,
  project_id      TEXT NOT NULL REFERENCES project(project_i
d) ON DELETE CASCADE,
  root_id         TEXT NOT NULL REFERENCES source_root(root_i
d) ON DELETE CASCADE,
  state           TEXT NOT NULL DEFAULT 'scanning'
                   CHECK (state IN ('scanning','importing','p
aused','completed','failed','cancelled')),
  started_at      TEXT NOT NULL,
  finished_at     TEXT,
  files_discovered INTEGER NOT NULL DEFAULT 0,
  files_imported   INTEGER NOT NULL DEFAULT 0,
  files_duplicate  INTEGER NOT NULL DEFAULT 0,
  files_quarantined INTEGER NOT NULL DEFAULT 0,
  bytes_hashed     INTEGER NOT NULL DEFAULT 0,
  app_version      TEXT NOT NULL,
  error_code       TEXT
) STRICT;

CREATE INDEX idx_import_project ON import_run(project_id, sta
rted_at DESC);

CREATE TABLE quarantine (
  quarantine_id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
  project_id  TEXT NOT NULL REFERENCES project(project_id) ON
DELETE CASCADE,
  import_id   TEXT REFERENCES import_run(import_id) ON DELETE
SET NULL,
  abs_path    TEXT NOT NULL,
  error_code  TEXT NOT NULL,
  detail      TEXT NOT NULL,
  attempts    INTEGER NOT NULL DEFAULT 1,
  first_at    TEXT NOT NULL,
  last_at     TEXT NOT NULL,
```

```
  resolved_at TEXT,
  UNIQUE (project_id, abs_path)
) STRICT;

CREATE INDEX idx_quarantine_open ON quarantine(project_id, re
solved_at) WHERE resolved_at IS NULL;

-----------------------------------------------------------------
---------
-- Job graph. Owned by aura-jobs, defined here because it mus
t migrate with
-- the catalog and because resume depends on it surviving a c
rash.
-----------------------------------------------------------------
---------
CREATE TABLE task (
  task_id       TEXT NOT NULL PRIMARY KEY,   -- '{stage}:{sub
ject}:{stage_version}'
  project_id    TEXT NOT NULL REFERENCES project(project_id)
ON DELETE CASCADE,
  run_id        TEXT NOT NULL,
  stage         TEXT NOT NULL,
  subject_id    TEXT NOT NULL,               -- photo_id, fil
e_id or import_id
  stage_version INTEGER NOT NULL,
  state         TEXT NOT NULL DEFAULT 'pending'
                 CHECK (state IN ('pending','ready','runnin
g','succeeded','failed','quarantined','skipped')),
  priority      INTEGER NOT NULL DEFAULT 100,
  attempts      INTEGER NOT NULL DEFAULT 0,
  max_attempts  INTEGER NOT NULL DEFAULT 3,
  cost_cpu_ms   INTEGER NOT NULL DEFAULT 0,
  cost_gpu_ms   INTEGER NOT NULL DEFAULT 0,
  cost_ram_mb   INTEGER NOT NULL DEFAULT 0,
  cost_vram_mb  INTEGER NOT NULL DEFAULT 0,
  lease_owner   TEXT,                        -- worker uuid
```

```
  lease_expires_at TEXT,
  heartbeat_at  TEXT,
  error_code    TEXT,
  error_detail  TEXT,
  created_at    TEXT NOT NULL,
  updated_at    TEXT NOT NULL,
  finished_at   TEXT
) STRICT;

CREATE INDEX idx_task_claimable ON task(project_id, state, pr
iority, created_at)
  WHERE state IN ('ready','pending');
CREATE INDEX idx_task_lease     ON task(state, lease_expires_
at) WHERE state = 'running';
CREATE INDEX idx_task_run        ON task(run_id, state);
CREATE INDEX idx_task_subject    ON task(subject_id, stage);

CREATE TABLE task_dep (
  task_id     TEXT NOT NULL REFERENCES task(task_id) ON DELET
E CASCADE,
  depends_on  TEXT NOT NULL REFERENCES task(task_id) ON DELET
E CASCADE,
  PRIMARY KEY (task_id, depends_on),
  CHECK (task_id <> depends_on)
) STRICT, WITHOUT ROWID;

CREATE INDEX idx_dep_reverse ON task_dep(depends_on);

-----------------------------------------------------------------
---------
-- Decision ledger. Empty in phase 01, created now because PH
ASE-13 must be
-- able to explain decisions made by earlier phases retrospec
tively.
-----------------------------------------------------------------
---------
```

```
CREATE TABLE decision_ledger (
  decision_id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
  project_id  TEXT NOT NULL REFERENCES project(project_id) ON
DELETE CASCADE,
  photo_id    TEXT REFERENCES photo(photo_id) ON DELETE CASCA
DE,
  run_id      TEXT NOT NULL,
  stage       TEXT NOT NULL,
  stage_version INTEGER NOT NULL,
  decision    TEXT NOT NULL,
  confidence  REAL,
  reason_json TEXT NOT NULL DEFAULT '{}',
  model_id    TEXT,
  decided_at  TEXT NOT NULL,
  reverted_at TEXT,
  reverted_by TEXT
) STRICT;

CREATE INDEX idx_decision_photo ON decision_ledger(photo_id,
stage);
CREATE INDEX idx_decision_run   ON decision_ledger(run_id, de
cided_at);

-----------------------------------------------------------------
---------
-- Settings, per catalog
-----------------------------------------------------------------
---------
CREATE TABLE setting (
  key        TEXT NOT NULL PRIMARY KEY,
  value_json TEXT NOT NULL,
  updated_at TEXT NOT NULL
) STRICT;

-----------------------------------------------------------------
---------
```

```
-- Convenience views. Views are safe to change without a migr
ation because
-- nothing persists through them.
-----------------------------------------------------------
---------
CREATE VIEW v_project_counts AS
SELECT p.project_id,
       p.name,
       COUNT(DISTINCT ph.photo_id) AS photo_count,
       COUNT(f.file_id)            AS file_count,
       COALESCE(SUM(f.size_bytes), 0) AS bytes_total
FROM project p
LEFT JOIN photo ph ON ph.project_id = p.project_id
LEFT JOIN photo_file f ON f.photo_id = ph.photo_id
WHERE p.deleted_at IS NULL
GROUP BY p.project_id;

INSERT INTO schema_version (version, applied_at, app_version,
migration_hash)
VALUES (1, :applied_at, :app_version, :migration_hash);
```

Design decisions worth defending

Decision	Reason	What it prevents
`STRICT` tables everywhere	SQLite otherwise accepts a string in an INTEGER column	A silent type drift discovered three phases later during a JOIN
`photo` separate from `photo_file`	RAW + camera JPEG + XMP are one photograph	Duplicate grid entries for every shot on RAW+JPEG cameras
Content hash unique per `(project, hash, rel_path)`	Re-import is a no-op; the same file copied twice is recorded once per location	Duplicate rows on the second import, the most common ingest bug
`fast_key` of size and mtime	Rescan skips rehashing unchanged files	Re-reading 220 GB to discover nothing changed

Decision	Reason	What it prevents
`is_present` rather than deleting rows	An unplugged drive is not a deletion	Losing ratings and edits because a USB cable was loose
Consent in its own table plus append-only ledger	Revocation must be provable	Unanswerable questions during a data-subject request
Enums as `TEXT` with `CHECK`	A support engineer can read the catalog	Guessing whether state 3 means paused or failed
`shutter_us` integer, not `REAL`	1/8000 as a float is not exact	Float comparison bugs in burst grouping in PHASE-08
`task` table in the catalog, not a separate DB	Resume state must be transactionally consistent with the data it describes	A completed task pointing at a photo row that was rolled back

2. Connection pragmas — `crates/aura-catalog/src/db.rs`

```rust
use std::path::Path;

use aura_core::{AuraResult, contract::error::{AuraError, Recovery, Severity}};
use rusqlite::Connection;

use crate::errors::{DB_OPEN_FAILED, DB_INTEGRITY};

/// Applied to every connection, read or write. Order matters: journal_mode
/// must be set before the first write, and busy_timeout before any contention.
pub fn apply_pragmas(conn: &Connection, writable: bool) -> AuraResult<()> {
    let statements: &[&str] = &[
        // Durability and concurrency
        "PRAGMA journal_mode = WAL",
```

```
        "PRAGMA synchronous = FULL",
        "PRAGMA foreign_keys = ON",
        "PRAGMA busy_timeout = 10000",
        // Performance
        "PRAGMA cache_size = -65536",        // 64 MB page ca
che
        "PRAGMA mmap_size = 268435456",      // 256 MB
        "PRAGMA temp_store = MEMORY",
        "PRAGMA wal_autocheckpoint = 2000",  // ~8 MB at 4 KB
pages
        // Safety
        "PRAGMA trusted_schema = OFF",
        "PRAGMA recursive_triggers = ON",
    ];
    for sql in statements {
        conn.execute_batch(sql).map_err(|e| AuraError::new(
            DB_OPEN_FAILED, Severity::Fatal, Recovery::AskUse
r,
            format!("pragma failed: {sql}"),
            "AURA could not prepare the catalog. Restart the
app; if it happens again, contact support.",
        ).with_cause(&e))?;
    }
    if !writable {
        conn.execute_batch("PRAGMA query_only = ON").map_err
(|e| AuraError::new(
            DB_OPEN_FAILED, Severity::Fatal, Recovery::Halt,
            "could not mark connection read-only",
            "AURA could not open the catalog safely.",
        ).with_cause(&e))?;
    }
    Ok(())
}
```

`synchronous = FULL`, **not** `NORMAL`. `NORMAL` is roughly 15 % faster on import and can lose the last transactions on power loss. A photographer importing a wedding at 2 a.m.

who loses power should lose the last file, not the last hour, and definitely not get a torn WAL. We spend the 15 % and get it back elsewhere by batching inserts.

`query_only = ON` **on reader connections.** The single-writer rule is enforced by the type system in Rust *and* by SQLite. Two independent mechanisms, because the cost of a second writer is catalog corruption.

3. Instance lock — `crates/aura-catalog/src/guard.rs`

```
use std::{fs::{File, OpenOptions}, path::Path};

use aura_core::{AuraResult, contract::error::{AuraError, Reco
very, Severity}};
use fs4::FileExt;

use crate::errors::DB_SECOND_WRITER;

/// Held for the lifetime of a writable catalog. Dropping it
releases the lock.
#[derive(Debug)]
pub struct CatalogLock {
    _file: File,
}

impl CatalogLock {
    /// Exclusive advisory lock on `<catalog>.lock`. We do no
t lock the catalog
    /// file itself: SQLite manages that, and taking our own
lock on it would
    /// fight with the WAL.
    pub fn acquire(catalog_path: &Path) -> AuraResult<Self> {
        let lock_path = catalog_path.with_extension("lock");
        let file = OpenOptions::new().create(true).read(tru
e).write(true)
            .open(&lock_path)
            .map_err(|e| AuraError::new(
```

```
                DB_SECOND_WRITER, Severity::RunBlocking, Reco
very::AskUser,
                "cannot create lock file",
                "AURA could not lock this catalog. Check fold
er permissions and try again.",
            ).with_cause(&e))?;

        file.try_lock_exclusive().map_err(|e| AuraError::new(
            DB_SECOND_WRITER, Severity::RunBlocking, Recover
y::AskUser,
            "catalog lock held by another process",
            "This catalog is already open in another AURA win
dow. Close it and try again.",
        ).with_cause(&e))?;

        Ok(Self { _file: file })
    }
}
```

An advisory lock file rather than a PID file: a PID file left behind by a crash blocks the next launch and generates a support ticket, whereas an OS-level advisory lock is released by the kernel when the process dies, including on `kill -9`. That single choice removes the "AURA says it is already running but it isn't" ticket class entirely.

4. Single-writer actor — `crates/aura-catalog/src/writer.rs`

```
use std::path::PathBuf;

use aura_core::AuraResult;
use crossbeam_channel::{bounded, Receiver, Sender};
use rusqlite::Connection;

/// A unit of work executed on the writer thread. Returning a
```

```
Result through a
/// oneshot channel keeps the caller's error handling complet
ely ordinary.
type WriteJob = Box<dyn FnOnce(&mut Connection) + Send>;

#[derive(Debug, Clone)]
pub struct Writer {
    tx: Sender<WriteJob>,
}

impl Writer {
    /// Spawn the one and only writer thread for this catalo
g.
    pub fn spawn(path: PathBuf) -> AuraResult<(Self, std::thr
ead::JoinHandle<()>)> {
        let (tx, rx): (Sender<WriteJob>, Receiver<WriteJob>)
= bounded(256);
        let mut conn = crate::db::open_raw(&path, true)?;
        let handle = std::thread::Builder::new()
            .name("aura-catalog-writer".into())
            .spawn(move || {
                while let Ok(job) = rx.recv() {
                    job(&mut conn);
                }
            })
            .map_err(|e| crate::errors::open_failed(&path, &
e))?;
        Ok((Self { tx }, handle))
    }

    /// Run a closure on the writer thread and wait for its r
esult.
    pub fn with<T, F>(&self, f: F) -> AuraResult<T>
    where
        T: Send + 'static,
        F: FnOnce(&mut Connection) -> AuraResult<T> + Send +
```

```
'static,
    {
        let (rtx, rrx) = bounded(1);
        self.tx.send(Box::new(move |conn| {
            let _ = rtx.send(f(conn));
        })).map_err(|_| crate::errors::writer_gone())?;
        rrx.recv().map_err(|_| crate::errors::writer_gone())?
    }

    /// Run a closure inside an IMMEDIATE transaction. Any er
ror rolls back.
    pub fn transact<T, F>(&self, f: F) -> AuraResult<T>
    where
        T: Send + 'static,
        F: FnOnce(&rusqlite::Transaction<'_>) -> AuraResult<T
> + Send + 'static,
    {
        self.with(move |conn| {
            let tx = conn.transaction_with_behavior(rusqlit
e::TransactionBehavior::Immediate)
                .map_err(|e| crate::errors::txn_failed(&e))?;
            let out = f(&tx)?;
            tx.commit().map_err(|e| crate::errors::txn_failed
(&e))?;
            Ok(out)
        })
    }
}
```

`TransactionBehavior::Immediate` takes the write lock at `BEGIN` rather than at the first write. With deferred transactions, two connections can both start, both read, and one gets `SQLITE_BUSY` on upgrade after doing all its work. Immediate turns that into an instant, cheap wait.

5. Migration runner — `crates/aura-catalog/src/migrate.rs`

```
use std::path::Path;

use aura_core::{AuraResult, clock::Clock};
use rusqlite::Connection;

pub const APP_SCHEMA_VERSION: i64 = 1;

/// Every migration is (version, name, sql). Embedded so a sh
ipped binary can
/// never disagree with its own migrations.
const MIGRATIONS: &[(i64, &str, &str)] = &[
    (1, "init", include_str!("../../../migrations/0001_init.s
ql")),
];

pub fn migrate(conn: &mut Connection, path: &Path, clock: &dy
n Clock, app_version: &str)
    -> AuraResult<()>
{
    let current = read_version(conn)?;

    // Refusal: a catalog from a newer app version must not b
e opened. Opening
    // it read-only and guessing would silently drop columns
we do not know about.
    if current > APP_SCHEMA_VERSION {
        return Err(crate::errors::schema_too_new(current, APP
_SCHEMA_VERSION));
    }
    if current == APP_SCHEMA_VERSION {
        return Ok(());
    }
```

```
    // Verified backup before any structural change. Verification means opening
    // the copy and running integrity_check, not merely checking the file exists.
    if current > 0 {
        let backup = crate::backup::create_verified(conn, path, current, clock)?;
        tracing::info!(target: "catalog", backup = %backup.display(), "pre-migration backup verified");
    }

    for (version, name, sql) in MIGRATIONS {
        if *version <= current { continue; }

        let hash = blake3::hash(sql.as_bytes()).to_hex().to_string();
        let tx = conn.transaction_with_behavior(rusqlite::TransactionBehavior::Immediate)
            .map_err(|e| crate::errors::migration_failed(*version, &e))?;

        tx.execute_batch(sql).map_err(|e| crate::errors::migration_failed(*version, &e))?;
        tx.execute(
            "INSERT OR REPLACE INTO schema_version (version, applied_at, app_version, migration_hash)
             VALUES (?1, ?2, ?3, ?4)",
            rusqlite::params![version, clock.now_utc().to_string(), app_version, hash],
        ).map_err(|e| crate::errors::migration_failed(*version, &e))?;

        tx.commit().map_err(|e| crate::errors::migration_failed(*version, &e))?;
        tracing::info!(target: "catalog", version, name, "mig
```

```
ration applied");
    }

    // Post-migration integrity check. A migration that leave
s a corrupt index
    // is worse than one that fails loudly.
    integrity_check(conn)?;
    Ok(())
}

fn read_version(conn: &Connection) -> AuraResult<i64> {
    let exists: i64 = conn.query_row(
        "SELECT COUNT(*) FROM sqlite_master WHERE type='tabl
e' AND name='schema_version'",
        [], |r| r.get(0),
    ).map_err(|e| crate::errors::open_failed_q(&e))?;
    if exists == 0 { return Ok(0); }
    conn.query_row("SELECT COALESCE(MAX(version), 0) FROM sch
ema_version", [], |r| r.get(0))
        .map_err(|e| crate::errors::open_failed_q(&e))
}

pub fn integrity_check(conn: &Connection) -> AuraResult<()> {
    let result: String = conn.query_row("PRAGMA integrity_che
ck", [], |r| r.get(0))
        .map_err(|e| crate::errors::integrity_unreadable(&
e))?;
    if result != "ok" {
        return Err(crate::errors::integrity_failed(&result));
    }
    let fk: i64 = conn.query_row("SELECT COUNT(*) FROM pragma
_foreign_key_check", [], |r| r.get(0))
        .unwrap_or(0);
    if fk > 0 {
        return Err(crate::errors::integrity_failed(&format!("
{fk} foreign key violations")));
```

```
    }
    Ok(())
}
```

6. Verified backup — `crates/aura-catalog/src/backup.rs`

```
use std::path::{Path, PathBuf};

use aura_core::{AuraResult, clock::Clock};
use rusqlite::{backup::Backup, Connection};

/// Create `backups/pre-<version>-<timestamp>.sqlite`, then open the copy and
/// verify it. An unverified backup is not a backup.
pub fn create_verified(conn: &Connection, catalog: &Path, from_version: i64, clock: &dyn Clock)
    -> AuraResult<PathBuf>
{
    let dir = catalog.parent().unwrap_or(Path::new(".")).join("backups");
    std::fs::create_dir_all(&dir).map_err(|e| crate::errors::backup_failed(&e))?;

    let stamp = clock.now_utc().unix_timestamp();
    let dest_path = dir.join(format!("pre-{from_version}-{stamp}.sqlite"));

    let mut dest = Connection::open(&dest_path).map_err(|e| crate::errors::backup_failed_q(&e))?;
    {
        let backup = Backup::new(conn, &mut dest).map_err(|e| crate::errors::backup_failed_q(&e))?;
        backup.run_to_completion(256, std::time::Duration::from_millis(1), None)
```

```
            .map_err(|e| crate::errors::backup_failed_q(&
e))?;
    }

    // Verify: integrity_check plus a row-count comparison on
the largest table.
    let check: String = dest.query_row("PRAGMA integrity_chec
k", [], |r| r.get(0))
        .map_err(|e| crate::errors::backup_failed_q(&e))?;
    if check != "ok" {
        let _ = std::fs::remove_file(&dest_path);
        return Err(crate::errors::backup_unverified(&check));
    }
    let src_files: i64 = conn.query_row("SELECT COUNT(*) FROM
photo_file", [], |r| r.get(0)).unwrap_or(-1);
    let dst_files: i64 = dest.query_row("SELECT COUNT(*) FROM
photo_file", [], |r| r.get(0)).unwrap_or(-2);
    if src_files != dst_files {
        let _ = std::fs::remove_file(&dest_path);
        return Err(crate::errors::backup_unverified("row coun
t mismatch"));
    }

    prune(&dir, 5);
    Ok(dest_path)
}

/// Keep the newest N backups. A disk full of backups is its
own failure.
fn prune(dir: &Path, keep: usize) {
    let mut entries: Vec<_> = std::fs::read_dir(dir).into_ite
r().flatten()
        .filter_map(Result::ok)
        .filter(|e| e.file_name().to_string_lossy().starts_wi
th("pre-"))
        .collect();
```

```
    entries.sort_by_key(|e| e.file_name());
    if entries.len() > keep {
        for old in &entries[..entries.len() - keep] {
            let _ = std::fs::remove_file(old.path());
        }
    }
}
```

7. Catalog open sequence — the whole refusal chain

```
// crates/aura-catalog/src/lib.rs (open)
pub fn open(path: &Path, clock: Arc<dyn Clock>, app_version:
&str) -> AuraResult<Catalog> {
    // 1. Never open a catalog inside a sync folder.
    aura_core::paths::assert_not_cloud_synced(path)?;
// AURA-IO-1008
    // 2. Exactly one writer per catalog, enforced by the OS.
    let lock = CatalogLock::acquire(path)?;
// AURA-DB-3005
    // 3. Open, apply pragmas, verify the file really is our
catalog.
    let mut conn = db::open_raw(path, true)?;
// AURA-DB-3001
    db::assert_is_aura_catalog(&conn)?;
// AURA-DB-3001
    // 4. Integrity before we trust a single row.
    migrate::integrity_check(&conn)?;
// AURA-DB-3003
    // 5. Migrate forward only, with a verified backup first.
    migrate::migrate(&mut conn, path, clock.as_ref(), app_ver
sion)?; // 3002/3004/3006
    // 6. Only now is a writer thread allowed to exist.
    drop(conn);
    let (writer, handle) = Writer::spawn(path.to_path_buf
())?;
```

```
    let readers = ReadPool::new(path, 4)?;
    Ok(Catalog { writer, readers, lock, handle, clock })
}
```

Six refusals before the first read. Each one is a code, a runbook and a test. This ordering is deliberate: the cloud-sync check is first because it is the cheapest and the most destructive if skipped, and the migration is last because it is the only step that writes.

8. Tests for T4–T6

```
// crates/aura-catalog/tests/schema.rs
use aura_catalog::*;

#[test]
fn fresh_catalog_reaches_current_version_and_is_clean() {
    let dir = tempfile::tempdir().expect("tmp");
    let cat = open(&dir.path().join("c.sqlite"), test_clock
(), "0.1.0-test").expect("open");
    assert_eq!(cat.schema_version().expect("v"), 1);
    cat.integrity_check().expect("integrity");
}

#[test]
fn strict_tables_reject_wrong_types() {
    let cat = fresh();
    let err = cat.writer().with(|c| Ok(c.execute(
        "INSERT INTO photo_file (file_id, photo_id, project_i
d, root_id, rel_path, file_name,
         ext, kind, role, size_bytes, content_hash, mtime, fa
st_key, first_seen_at, last_seen_at)
         VALUES ('f','p','pr','r','a','a','cr2','raw','primar
y','not-a-number','h','t','k','t','t')",
        [])))).expect("call");
    assert!(err.is_err(), "STRICT must reject a text size_byt
```

```
es");
}

#[test]
fn second_instance_is_refused_with_3005() {
    let dir = tempfile::tempdir().expect("tmp");
    let p = dir.path().join("c.sqlite");
    let _first = open(&p, test_clock(), "0.1.0").expect("firs
t open");
    let second = open(&p, test_clock(), "0.1.0");
    let err = second.err().expect("must fail");
    assert_eq!(err.code.0, "AURA-DB-3005");
    assert!(err.user_message.contains("another AURA windo
w"));
}

#[test]
fn cloud_synced_path_is_refused_with_1008() {
    let dir = tempfile::tempdir().expect("tmp");
    let p = dir.path().join("Dropbox").join("c.sqlite");
    std::fs::create_dir_all(p.parent().expect("parent")).expe
ct("mkdir");
    let err = open(&p, test_clock(), "0.1.0").err().expect("m
ust fail");
    assert_eq!(err.code.0, "AURA-IO-1008");
}

#[test]
fn newer_schema_is_refused_not_downgraded() {
    let cat = fresh();
    cat.writer().with(|c| Ok(c.execute(
        "INSERT INTO schema_version VALUES (99, '2026-01-01T0
0:00:00Z', '9.9.9', 'x')", []))).expect("i");
    drop(cat);
    let err = reopen().err().expect("must refuse");
    assert_eq!(err.code.0, "AURA-DB-3004");
```

```
}

#[test]
fn corrupt_catalog_is_detected_before_any_read() {
    let path = fresh_path();
    { let _c = open(&path, test_clock(), "0.1.0").expect("ope
n"); }
    // Scribble over the middle of the file, then reopen.
    use std::io::{Seek, SeekFrom, Write};
    let mut f = std::fs::OpenOptions::new().write(true).open
(&path).expect("open raw");
    f.seek(SeekFrom::Start(8192)).expect("seek");
    f.write_all(&[0xAB; 2048]).expect("scribble");
    let err = open(&path, test_clock(), "0.1.0").err().expect
("must fail");
    assert!(matches!(err.code.0, "AURA-DB-3003" | "AURA-DB-30
01"));
}

#[test]
fn migration_failure_leaves_a_verified_backup() {
    // Inject a migration that fails halfway, assert the cata
log still opens at
    // the old version and the backup exists and passes integ
rity_check.
}

#[test]
fn writer_is_single_threaded_under_contention() {
    let cat = fresh();
    let w = cat.writer().clone();
    let handles: Vec<_> = (0..16).map(|i| {
        let w = w.clone();
        std::thread::spawn(move || w.transact(move |tx| {
            tx.execute("INSERT INTO setting VALUES (?1, '1',
'2026-01-01T00:00:00Z')",
```

```
                rusqlite::params![format!("k{i}")]).map_err(|
e| test_err(&e))?;
            Ok(())
        }))
    }).collect();
    for h in handles { h.join().expect("join").expect("txn");
}
    assert_eq!(cat.count("setting").expect("count"), 16);
}
```

9. Acceptance criteria for T4–T6

- [ ] `0001_init.sql` applied to an empty file yields schema version 1 and `PRAGMA integrity_check` returns `ok`
- [ ] Every table is `STRICT`; the wrong-type insert test fails as expected
- [ ] `PRAGMA foreign_key_check` returns zero rows on a fixture-loaded catalog
- [ ] Opening a catalog in a folder named `Dropbox` fails with `AURA-IO-1008`; renaming the folder but leaving a `.dropbox` file still fails
- [ ] Two `open()` calls on one path: the second fails with `AURA-DB-3005`; after `kill -9` of the first process the lock is free immediately, proven by test
- [ ] A catalog with `schema_version = 99` is refused with `AURA-DB-3004` and is left byte-identical afterwards
- [ ] A deliberately corrupted catalog is refused with `AURA-DB-3003` before any row is read
- [ ] A failing migration restores the pre-migration state and leaves a backup that itself passes `integrity_check`
- [ ] Backups prune to the newest 5
- [ ] 16 concurrent writer calls all commit; no `SQLITE_BUSY` surfaces to the caller
- [ ] `EXPLAIN QUERY PLAN` for the four hot queries (claimable tasks, files by hash, photos by capture time, open quarantine) shows an index scan, not a table scan; output pasted

- [ ] Catalog open on a 200,000-row fixture measured under 600 ms

10. Claude Code prompt for T4–T6

> Act as `SRC` (Senior Rust, Core). Implement T4, T5 and T6 of PHASE-01. Copy `migrations/0001_init.sql` **verbatim** from this page — do not add, rename or reorder a single column, and do not "optimise" the indices. Then implement `db.rs` pragmas, `guard.rs` advisory lock, `writer.rs` single-writer actor, `migrate.rs`, `backup.rs`, the six-step `open()` refusal chain, and every test in section 8 including the two that are only sketched.

> Hard constraints: exactly one writable connection per catalog; reader connections must be `query_only`; no `unwrap`; every failure path returns one of the DB or IO codes already registered in `errors.toml`; no new error code without adding it to the registry and writing its runbook.

> Evidence I expect pasted back: `cargo test -p aura-catalog` in full; the `EXPLAIN QUERY PLAN` output for the four hot queries; the `kill -9` lock-release demonstration; the corrupt-catalog and newer-schema refusals showing the exact error codes; and the 200,000-row open timing. Stop after T6.